How a scikit-learn Pipeline Works

A beginner-friendly explanation of this code

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline(steps=[
    ("missing_value_handler", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)
```

1 Input Data

X_train

Age	Salary	Score
25	50000	80
NaN	62000	NaN
45	90000	70
30	NaN	90

```
y_train
[1, 0, 1, 0]
```

X_train contains the training features.

y_train contains the correct target values.

Example:

Age, Salary, Score

↓

Churn / No Churn

2 Missing Value Handler

```
SimpleImputer
(strategy='median')
```

Fills missing numeric values using the median of each column.

[10, 12, NaN, 20]

↓

median = 12

↓

[10, 12, 12, 20]

Why? ML models cannot learn from blank values.

3 Scaling

```
StandardScaler()
```

Standardizes numeric features so they are on a similar scale.

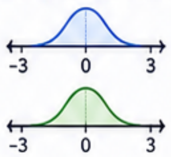
Before (different ranges)

Age: 18 – 60

Salary: 20000 – 90000

↓

After (standardized)

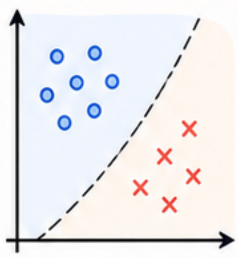


Why? Features with very different ranges can affect model learning.

4 Model

```
LogisticRegression()
```

Learns patterns from the processed training data and predicts a class such as 0/1 or Yes/No.



Despite the name, Logistic Regression is used for classification.

5 Training the Pipeline

```
pipeline.fit(
X_train, y_train)
```

This single command runs all steps in order:

- fill missing values
- scale the features
- train the logistic regression model

You do not need to call each step manually.

Why use a Pipeline?

Reusable: same workflow can be run again.

Cleaner code: all steps in one place.

Consistent: same preprocessing every time.

Less error-prone: fewer manual mistakes.

Beginner Tip: This simple pipeline works best when all input features are numeric. If your dataset also has text or categorical columns, you usually combine pipelines with a **ColumnTransformer**.